PatternWise — Final Submission Supporting Notes

Mahalakshmi V | 2nd Year, CSE - B

GitHub: https://github.com/MahalakshmiV10/PatternWise

Pattern Selection Checklist

PatternWise — Pattern-Selection Checklist

Use this checklist to select a pattern based on the design problem. A pattern is a reusable approach, not a rule that must be used in every project.

■ **Singleton** — Is one shared instance required to coordinate access to a resource?

■ **Factory Method** — Should object creation be delegated to subclasses or a creation method?

■ **Builder** — Does an object have many optional parts or a step-by-step construction process?

■ **Adapter** — Do two interfaces need to work together despite incompatible APIs?

■ **Decorator** — Must behavior be added to an object dynamically without changing its class?

■ **Facade** — Would a simple interface help hide a complicated subsystem?

■ **Strategy** — Should an algorithm be interchangeable at runtime?

■ **Observer** — Should multiple listeners be notified when one object's state changes?

Selection questions

1. What problem is being solved?

2. Which part changes most often?

3. Is object creation, interface compatibility, behavior extension, subsystem complexity, algorithm choice, or event notification the main issue?

4. Would the pattern reduce complexity, or add unnecessary abstraction?

5. Can the choice be explained with a small example and tested?

Pattern Notes

PatternWise — Pattern Notes

1. Singleton

Intent: Ensure a class has one shared instance and provide a common access point.

Use when: A genuinely shared coordinator or resource is required.

Watch out: Global shared state can make testing and concurrency harder.

Example idea: One application configuration manager.

2. Factory Method

Intent: Let a creation method decide which concrete product to return.

Use when: Client code should depend on a common product interface rather than concrete classes.

Watch out: Too many product types can increase class complexity.

Example idea: A notification creator returns email or SMS notification objects.

3. Builder

Intent: Construct a complex object step by step.

Use when: Many optional settings or construction steps would make a constructor confusing.

Watch out: A builder may be excessive for a simple object.

Example idea: Configure a report with title, sections, and output format.

4. Adapter

Intent: Convert one interface into another interface the client expects.

Use when: Integrating legacy code or a third-party API with a different interface.

Watch out: Too many adapters can obscure the underlying API.

Example idea: Adapt a legacy temperature service to a standard temperature interface.

5. Decorator

Intent: Add responsibilities to an object by wrapping it.

Use when: Features should be combined flexibly without creating many subclasses.

Watch out: Several wrappers can make execution order harder to trace.

Example idea: Add logging or compression around a data stream.

6. Facade

Intent: Provide a simple entry point to a complex subsystem.

Use when: Clients should not need to coordinate many subsystem components.

Watch out: Avoid turning the facade into an oversized class containing all logic.

Example idea: A home-theatre facade starts the display, speakers, and player.

7. Strategy

Intent: Encapsulate alternative algorithms behind a common interface.

Use when: The algorithm should be chosen or changed without rewriting the client.

Watch out: The client or configuration still needs to select a strategy.

Example idea: Choose standard, express, or pickup delivery calculation.

8. Observer

Intent: Notify subscribed observers when a subject changes.

Use when: One event should update multiple independent listeners.

Watch out: Manage subscriptions and avoid unexpected notification chains.

Example idea: Notify dashboard widgets when a data source updates.

Comparison

| Pattern | Main design concern | Typical benefit |

|---|---|---|

| Singleton | Shared instance | Centralized access |

| Factory Method | Object creation | Reduced coupling to concrete products |

| Builder | Construction | Clear step-by-step setup |

| Adapter | Interface mismatch | Compatibility |

| Decorator | Add-on behavior | Flexible composition |

| Facade | Subsystem complexity | Simpler client interface |

| Strategy | Algorithm variation | Easy algorithm substitution |

| Observer | Event notification | Loose coupling between publisher and listeners |

Quiz and Answer Key

PatternWise — Quiz and Answer Key

Questions

1. Which pattern is intended to provide one shared instance?
2. Which pattern delegates the decision of which concrete product to create?
3. Which pattern is useful for step-by-step construction of a complex object?
4. Which pattern converts one interface into another expected interface?
5. Which pattern adds behavior by wrapping an existing object?
6. Which pattern provides a simplified entry point to a subsystem?
7. Which pattern allows an algorithm to be swapped independently?
8. Which pattern notifies multiple subscribers when a subject changes?
9. Give one potential drawback of Singleton.
10. Why should a developer avoid using a pattern when the problem does not need it?

Answer key

1. Singleton.
2. Factory Method.
3. Builder.
4. Adapter.
5. Decorator.
6. Facade.
7. Strategy.
8. Observer.
9. Shared global state can complicate tests and concurrency.
10. Unnecessary abstractions increase complexity and make code harder to maintain.

Before After Analysis

PatternWise — Before/After Analysis

This is a qualitative design comparison to be validated against the actual examples in the repository.

| Situation | Before applying a suitable pattern | After applying a suitable pattern | Expected effect |

|---|---|---|---|

| Shared service | Multiple instances may be created independently | A shared instance can be accessed through Singleton | Consistent access; shared-state trade-offs remain |

| Product creation | Client code may depend directly on concrete classes | Factory Method centralizes or delegates creation | Lower coupling |

| Complex object setup | Many constructor parameters or setup steps | Builder separates construction steps | Clearer configuration |

| Incompatible API | Client code must understand a foreign interface | Adapter translates the interface | Easier integration |

| Optional features | Subclasses multiply for combinations of features | Decorators wrap objects with extra behavior | Flexible composition |

| Multi-part subsystem | Client coordinates many components | Facade offers a simpler entry point | Simpler client code |

| Changing algorithms | Conditional logic selects behavior throughout client code | Strategy encapsulates alternatives | Easier substitution |

| Multiple event listeners | Publisher may directly call specific listeners | Observer uses subscription/notification | Looser coupling |

Conclusion

Design patterns can improve maintainability when they address a real design problem. They are not automatically better in every case; the implementation should remain understandable and testable.

Final Demo Checklist

PatternWise — Final Demo Checklist

Before the demo

■ Pull the final code from GitHub.

■ Install requirements and run the test suite.

■ Start the Flask app and confirm the local URL opens.

■ Close unrelated tabs and keep the browser zoom readable.

■ Keep a backup screenshot or short recording in case the live environment fails.

Suggested 5–7 minute flow

1. **Introduction (30 sec):** State the goal of PatternWise.

2. **Home page (45 sec):** Show navigation and how patterns are organized.

3. **Pattern walkthrough (3 min):** Explain the problem and example for Singleton, Factory Method, Builder, Adapter, Decorator, Facade, Strategy, and Observer briefly.

4. **Quiz / learning interaction (45 sec):** Demonstrate it if available in the submitted version.

5. **Technical overview (45 sec):** Mention Flask, HTML/CSS/templates, project structure, and GitHub.

6. **Testing (30 sec):** Show the real test command and current output.

7. **Conclusion (20 sec):** Summarize what was learned and possible future improvements.

Closing statement

“PatternWise is an interactive learning guide that introduces eight common software design patterns through concise explanations and examples. It is intended to help learners connect design problems with reusable solutions.”